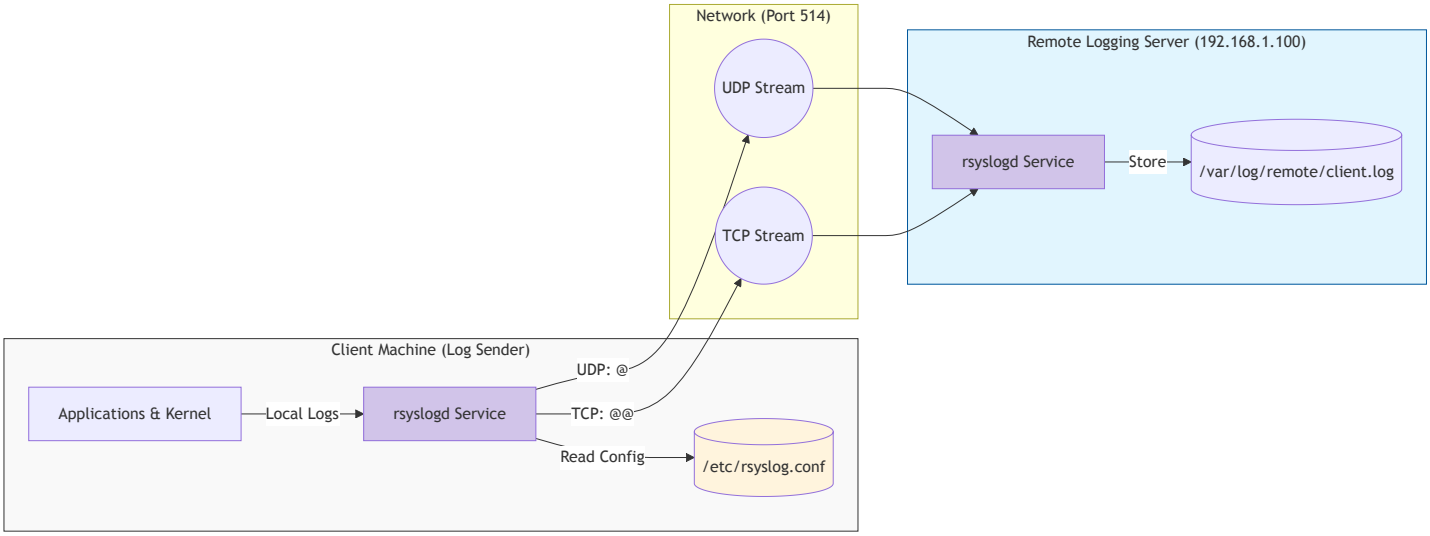
Exercise 4 - Configure rsyslog to send logs to a remote server

Question:

```
Exercice 4 – Configurer rsyslog pour envoyer des
journaux à un serveur distant

Configurer un serveur Unix pour envoyer tous
les messages de journal à un serveur distant ayant
l'adresse IP 192.168.1.100 sur le port UDP 514.
```

Answer:

```
Ouvrez /etc/rsyslog.conf sur le client et ajoutez la ligne
suivante pour envoyer les journaux au serveur distant :

*.* @192.168.1.100:514


Si vous voulez utiliser TCP (plus fiable qu'UDP qui ne
garanti pas la livraison des messages) pour plus de
fiabilité, utilisez @@ au lieu de @ :

*.* @@192.168.1.100:514
```



This exercise focuses on centralizing log management by configuring a Unix/Linux system (the client) to forward all its log messages to a dedicated remote server. Centralized logging is crucial for

security analysis, auditing, and troubleshooting multiple systems from a single location.

Detailed Explanation:

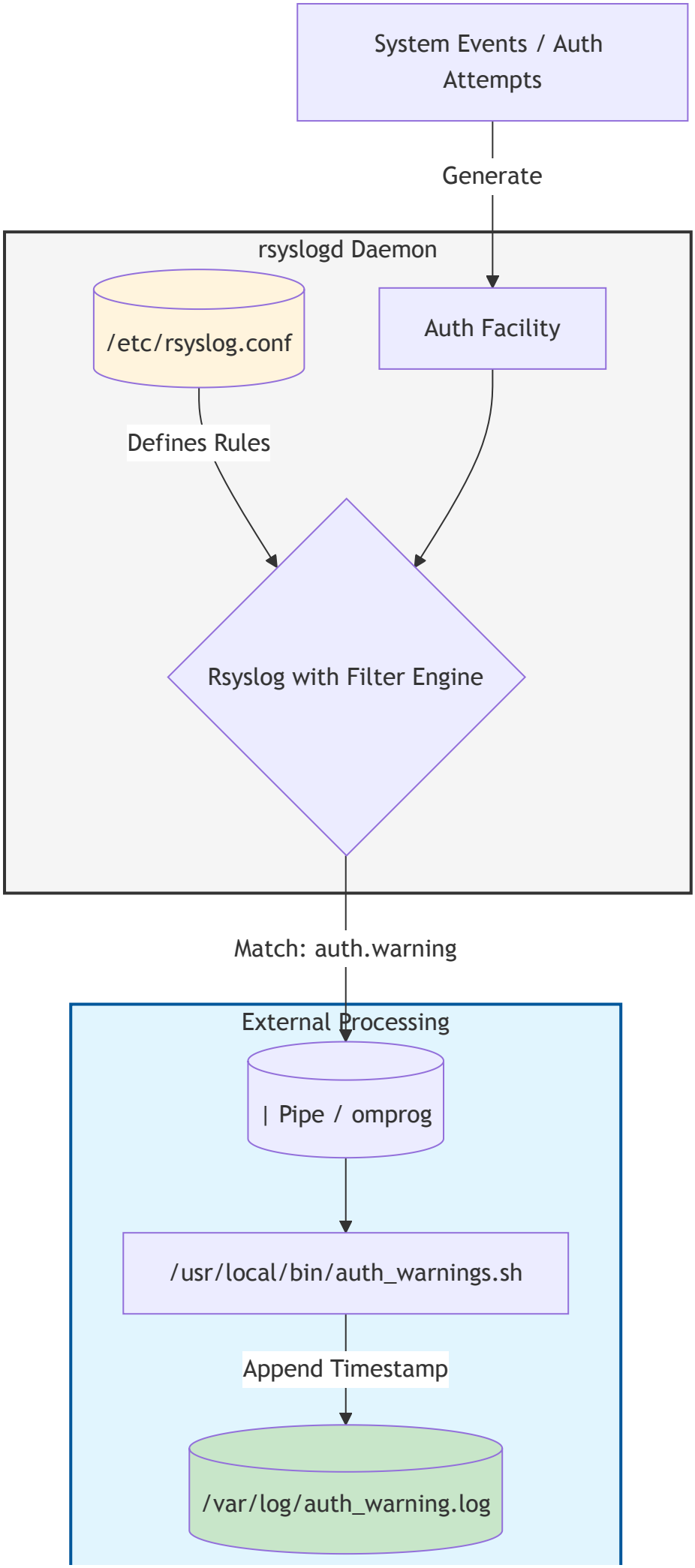
1. **Objective:** The primary goal is to redirect the local system's logging stream. Instead of (or in addition to) writing logs to files like `/var/log/syslog` locally, the system's `rsyslog` daemon will transmit each log entry over the network to a server at IP address `192.168.1.100`.
2. **Configuration File:** The configuration for the `rsyslog` daemon is primarily stored in `/etc/rsyslog.conf`. This file contains rules that define what log messages (based on their facility and severity) go to which destination (file, console, remote host, etc.).
3. **The Forwarding Rule Syntax:** The line `*.* @192.168.1.100:514` is a forwarding rule.
   - `*.*` : The first `*` represents the **facility** (the subsystem generating the message, e.g., `auth`, `kern`, `mail`). The second `*` represents the **severity** or priority (e.g., `emerg`, `alert`, `crit`, `err`, `warning`, `notice`, `info`, `debug`). Using `*.*` means "ALL facilities at ALL severity levels."
   - `@` : The `@` symbol is the protocol indicator. A single `@` signifies transmission using the **UDP** (User Datagram Protocol).
   - **`192.168.1.100:514`** : This is the destination. `192.168.1.100` is the remote server's IP address, and `514` is the standard port for the syslog protocol.
4. **UDP vs. TCP – A Critical Distinction:**
   - **UDP (`@`)**: As stated, UDP does not guarantee delivery. It is a connectionless, "fire-and-forget" protocol. This means log messages can be lost due to network congestion without the sender knowing. It is faster and has less overhead but is unreliable. The configuration `*.* @192.168.1.100:514` uses UDP.
   - **TCP (`@@`)**: Using a double `@` (`@@`) instructs `rsyslog` to use **TCP** (Transmission Control Protocol). TCP is connection-oriented. It establishes a session with the server and guarantees delivery by requesting acknowledgments and retransmitting lost packets. This is essential for reliable log collection where missing an "authentication failure" or "disk full" message is unacceptable. The line `*.* @@192.168.1.100:514` would enable this more reliable method.
5. **Steps:**
   - Edit the client's configuration file: `sudo nano /etc/rsyslog.conf`
   - Add the appropriate line (`@` for UDP, `@@` for TCP) to the end of the rules section.
   - Restart the `rsyslog` service to apply changes: `sudo systemctl restart rsyslog`

Exercise 5 - Filter "warning" level messages and pipe them to a Bash script

Question:

```
Exercice 5 - Filtrer les messages de niveau “warning”
et les transmettre à un script Bash pour traitement

Configurer rsyslog pour détecter tous les messages de
niveau warning (ou supérieur) provenant de la facilité
auth et les envoyer à un script Bash. Ce script doit
enregistrer les messages dans un fichier
/var/log/auth_warning.log avec un horodatage du type :
2024-12-30 14:35:42 Authentication failure for user 'admin’.
```

Answer:


System Events / Auth Attempts
Generate
rsyslogd Daemon
/etc/rsyslog.conf
Auth Facility
Defines Rules
Rsyslog with Filter Engine
Match: auth.warning
External Processing
| Pipe / omprog
/usr/local/bin/auth_warnings.sh
Append Timestamp
/var/log/auth_warning.log

Créez un fichier de script nommé `/usr/local/bin/auth_warnings.sh` :

```
#!/bin/bash
while read -r line; do
    echo "$(date '+%Y-%m-%d %H:%M:%S') $line" >> /var/log/auth_warning.log
Done
```

Rendez-le exécutable :

```
sudo chmod +x /usr/local/bin/auth_warnings.sh
```

Modifiez le fichier /etc/rsyslog.conf :

```
auth.warning | /usr/local/bin/auth_warnings.sh
```

Ou

```
if ($syslogfacility-text == 'auth' and $msg contains
'warning') then | /usr/local/bin/auth_warnings.sh
```

Ou

```
if ($syslogfacility-text == 'auth' and $syslogseverity <= 4) then {
    action(type="omprog" binary="/usr/local/bin/auth_warnings.sh")
    stop
    }
```

This exercise demonstrates advanced log processing by filtering specific messages and passing them to a custom script for specialized handling, rather than just writing them to a file.

Detailed Explanation:

1. **Objective:** Intercept all log messages from the `auth` / `authorization` facility (which handles login, sudo, ssh, etc.) that have a severity of `warning` or higher (i.e., more severe: `err`, `crit`, `alert`, `emerg`). These filtered messages are then streamed to a custom Bash script for formatting and output.
2. **Understanding Facilities and Severities:** Rsyslog messages have two key properties:

- **Facility:** The source of the message (e.g., `kern` for kernel, `mail` for mail system, `auth` for security/authentication).
- **Severity:** The urgency/importance, numbered 0-7 (0=emerg, 1=alert... 4=warning, 5=notice, 6=info, 7=debug). *Lower numbers are more severe.*

3. **The Bash Script (`/usr/local/bin/auth_warnings.sh`):**
   - This script acts as a simple processor. It runs in a loop, continuously reading (`read -r line`) from its standard input (`stdin`), which will be connected to `rsyslog` .
   - For each line (log message) it receives, it prepends a precise timestamp (`$(date '+%Y-%m-%d %H:%M:%S')`) and appends the entire formatted line to the file `/var/log/auth_warning.log` .
   - The `chmod +x` command makes the script executable.
4. **Configuring Rsyslog to Use the Script:**
   Three methods are shown, with increasing complexity and power:
   - **Method 1: Traditional Pipe Syntax (`|`)**

     ```
     auth.warning | /usr/local/bin/auth_warnings.sh
     ```

     This is the classic syntax. It pipes all `auth` facility messages of severity `warning` (4) and above to the script. The script receives the raw message.
   - **Method 2: Legacy Property-Based Filter in `rsyslog.conf`**

     ```
     if ($syslogfacility-text == 'auth' and $msg contains 'warning') then |/usr/loca
     ```

     This uses an `if` statement with message properties. It filters for messages where the facility text is 'auth' **AND** the message text contains the substring 'warning'. This is less precise than using severity levels, as it filters on text, not the intrinsic severity property.
   - **Method 3: Modern RainerScript Syntax with `omprog` Module (Recommended)**

     ```
     if ($syslogfacility-text == 'auth' and $syslogseverity <= 4) then {
         action(type="omprog" binary="/usr/local/bin/auth_warnings.sh")
         stop
     }
     ```

     This is the most powerful and modern approach.
     - **`if (...)`** : The condition checks for `auth` facility and a `syslogseverity` property value less than or equal to 4 (`warning` =4, `err` =3, etc.).
     - **`action(type="omprog" ...)`** : Uses the `omprog` (output module for program) module to execute the specified binary/script. This is more robust and feature-rich than the simple pipe.
     - **`stop`** : This directive is crucial. It means that if a message matches this rule, processing for it stops here. It will not be written to the default system log files (like

`/var/log/syslog`) unless another rule without a `stop` command catches it earlier. This prevents duplicate log entries.

omprog is the modern, reliable way for Rsyslog to send log messages to an external script or program.

Think of it as a **supercharged replacement for the simple pipe (`|`)**. Here's why it's better:

- **Reliable**: It uses a confirmation system. Your script must send an "acknowledgment" back to Rsyslog after processing each message, guaranteeing the log isn't lost.
- **Efficient**: It runs your script as a **single, long-running process**, streaming all messages to it. This is much faster than starting a new script instance for every log line (which the old pipe method does).
- **Controllable**: It gives you fine-grained control over timeouts, restarts, and message formatting.

In short: For any important log processing in production, use `omprog` . The simple pipe (`|`) is only suitable for basic testing.

More on `omprog` :

1. Omprog is an output module for the rsyslog daemon on Linux.
2. Its name stands for "Output Module Program."
3. This module forwards log messages to an external application.
4. The external program receives messages via its standard input (stdin).
5. Omprog enables real-time log processing and analysis.
6. You must configure it within an rsyslog ruleset.
7. Configuration uses the `omprog` action command.
8. You must specify the full path to your program or script.
9. A typical configuration line might be: `action(type="omprog" ...)` .
10. The target program, like a Python script, runs continuously.
11. Each new syslog message is piped to this waiting process.
12. This allows for immediate reaction to specific log events.
13. Omprog is more dynamic than simply writing to a flat file.
14. It is perfect for creating custom alerts and notifications.
15. For example, a script could parse a "failure" message and send an email.
16. The program must be designed to handle a continuous stream of data.
17. Proper signal handling in the script is important for clean shutdowns.
18. Using Omprog, you can integrate logs with other IT systems.
19. It transforms rsyslog from a passive recorder to an active tool.
20. Therefore, Omprog is a powerful tool for automation and monitoring.

Exercise 6 – Monitor logs for a specific user

Question:

```
Exercice 6 – Surveiller les journaux pour un utilisateur

Créer un script qui surveille des journaux pour un utilisateur spécifique
1. Vérifie si un utilisateur donné existe sur le système.
2. Recherche dans les journaux (/var/log/syslog) des
 événements contenant le nom de l'utilisateur.
3. Enregistre les résultats dans un fichier texte situé
dans le répertoire personnel de l'utilisateur.
4. Planifie l'exécution du script quotidiennement avec cron.
```

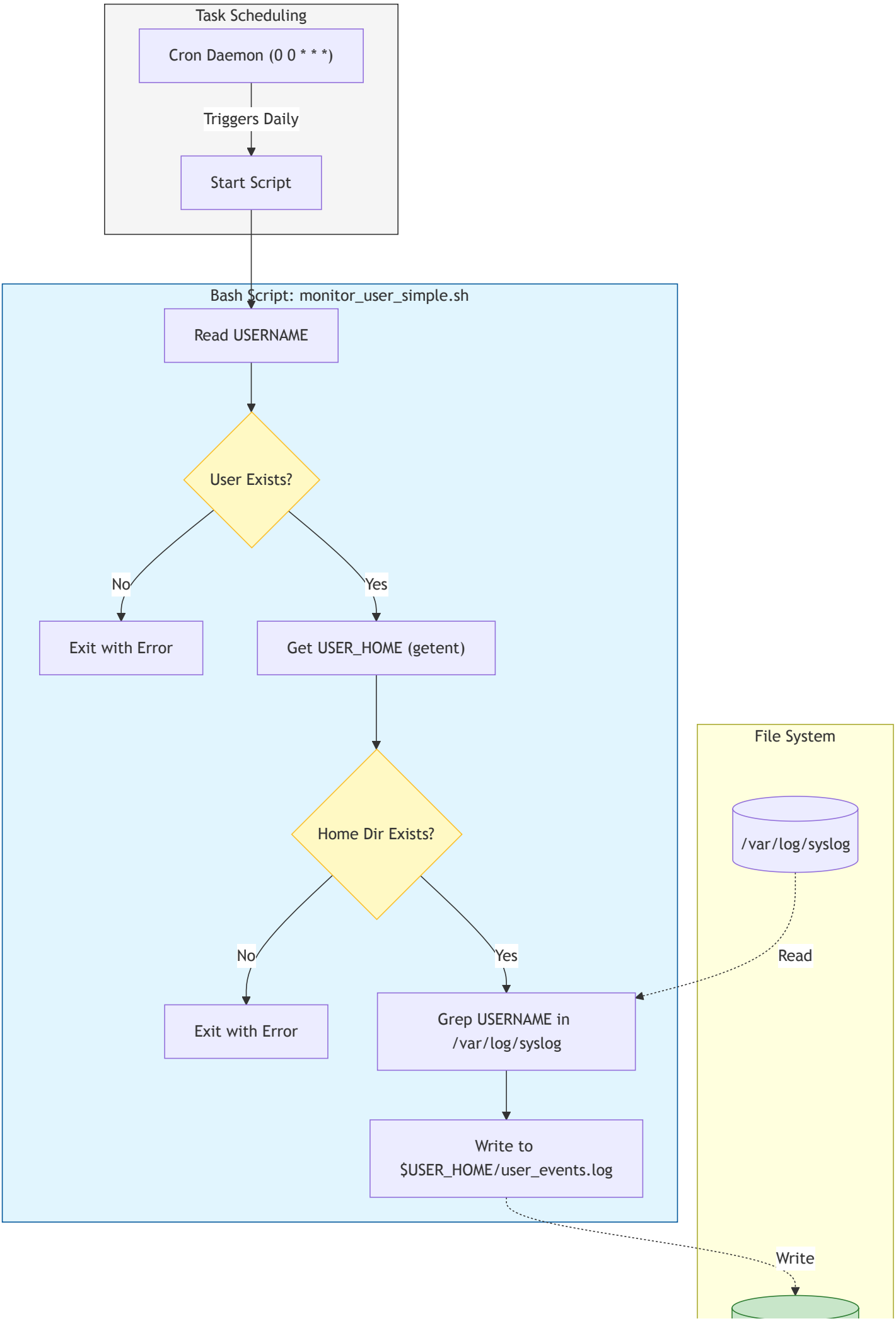
Answer:

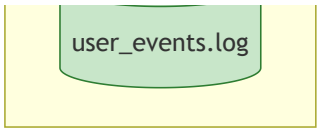
SCRIPT DE SURVEILLANCE

```bash
#!/bin/bash
# Demander le nom de l'utilisateur
echo "Entrez le nom de l'utilisateur à surveiller :"
read -r USERNAME

# Vérifier si l'utilisateur existe
if ! id "$USERNAME" &>/dev/null; then
    echo "L'utilisateur $USERNAME n'existe pas."
    exit 1
Fi

# Répertoire de l'utilisateur
USER_HOME=$(getent passwd "$USERNAME" | cut -d: -f6)
# getent passwd "$USERNAME" recherche l'entrée correspondant à l'utilisateur $USERNAME
# dans la base de données des utilisateurs (login:passwd:uid:gid:gecos:home dir:shell)

# Vérifier si le répertoire utilisateur existe
if [ ! -d "$USER_HOME" ]; then
    echo "Le répertoire de l'utilisateur $USERNAME est introuvable."
    exit 1
Fi

# Fichier pour enregistrer les événements
EVENTS_FILE="$USER_HOME/user_events.log"
# Rechercher les événements contenant le nom de l'utilisateur dans /var/log/syslog
echo "Événements pour l'utilisateur $USERNAME (exécuté le $(date)) :" > "$EVENTS_FILE"
grep "$USERNAME" /var/log/syslog >> "$EVENTS_FILE »

echo "Les événements ont été enregistrés dans $EVENTS_FILE"
```

PLANNIFICATION DE L'EXECUTION

```bash
crontab -e

0 0 * * * /home/utilisateur/scripts/monitor_user_simple.sh >
        /home/utilisateur/scripts/monitor_user_simple.log 2>&1
```


Task Scheduling
Cron Daemon (0 0 * * *)
Triggers Daily
Start Script
Bash Script: monitor_user_simple.sh
Read USERNAME
User Exists?
No
Yes
Exit with Error
Get USER_HOME (getent)
Home Dir Exists?
No
Yes
Exit with Error
Grep USERNAME in /var/log/syslog
Write to $USER_HOME/user_events.log
File System
/var/log/syslog
Read
Write



This exercise combines shell scripting and task scheduling (`cron`) to create an automated monitoring tool for a specific user's activity.

Detailed Explanation of the Script (`monitor_user_simple.sh`):

1. **Objective:** Create a script that, when run, searches all system logs for entries related to a given username and saves them to a dated report file within that user's home directory. Then, schedule this script to run automatically every day.
2. **Script Walkthrough:**
   - **User Input:** `read -r USERNAME` prompts the script runner to input a username.
   - **User Existence Check:** `id "$USERNAME" &>/dev/null` checks if the user exists on the system. If the `id` command fails (non-zero exit code), the script prints an error and exits.
   - **Find User's Home Directory:** `getent passwd "$USERNAME" | cut -d: -f6` is a robust way to get the user's home directory path from the system's user database. `getent` works with various user backends (local files, LDAP). `cut` extracts the 6th field (the home directory) from the colon-separated output.
   - **Home Directory Check:** Verifies the extracted home directory path actually exists.
   - **Define Output File:** Sets the output file path to `$USER_HOME/user_events.log` .
   - **Search Logs:** The core command is `grep "$USERNAME" /var/log/syslog` . It searches the main system log for any line containing the username and appends (`>>`) it to the output file. A header line with the execution date is written first (`>` overwrites).
   - **Note:** This script only checks `/var/log/syslog` . A more robust version would also grep files in `/var/log/` like `auth.log` , `secure` , `messages` , etc.
3. **Scheduling with Cron:**
   - `crontab -e` edits the current user's cron table, a list of scheduled jobs.
   - The line `0 0 * * * /path/to/script.sh > /path/to/logfile.log 2>&1` means:
     - **`0 0 * * *`** : The schedule. Minute=0, Hour=0, Day-of-month=*, Month*=, Day-of-week=*. This translates to "At 00:00 (midnight) every day, every month, on every day of the week."
     - **`/home/utilisateur/scripts/monitor_user_simple.sh`** : The absolute path to the script to execute.
     - **`> /home/utilisateur/scripts/monitor_user_simple.log 2>&1`** : This part handles output redirection.
       - `>` redirects the script's standard output (`stdout`) to the `.log` file.
       - `2>&1` redirects standard error (`stderr` , file descriptor 2) to the same place as `stdout` (file descriptor 1). This ensures both normal output and any error

messages from the script are captured in the log file, which is vital for debugging cron jobs.

https://crontab.guru/

https://www.redhat.com/en/blog/linux-cron-command

1. Motivation: Why `cron` ?

In many systems, we need tasks to run **automatically at specific times**:

- Backup databases every night
- Clean temporary files every hour
- Run ML training scripts periodically
- Monitor system health

Instead of manually executing commands, Linux provides a built-in scheduler:

> **`cron` — a time-based job scheduler**

The interface we use to configure it is:

> **`crontab` (cron table)**

2. Conceptual Model

Think of `cron` as a loop running in the background:

```
while true:
    check current time
    if matches any scheduled job:
        execute command
```

Each user has their own **crontab file**, which defines:

- *when* to run
- *what* to run

3. Basic Commands

View current crontab

```
crontab -l
```

Edit crontab

```
crontab -e
```

Remove crontab

```
crontab -r
```

Edit another user's crontab (requires privilege)

```
sudo crontab -u username -e
```

4. Crontab Syntax

Each line in a crontab file has the form:

```
* * * * * command
| | | | |
| | | | └── Day of week (0–7) (Sunday = 0 or 7)
| | | └──── Month (1–12)
| | └────── Day of month (1–31)
| └──────── Hour (0–23)
└────────── Minute (0–59)
```

5. Examples

Run a script every day at 2:30 AM

```
30 2 * * * /home/user/backup.sh
```

Run every 5 minutes

```
*/5 * * * * /home/user/script.sh
```

Run every Monday at 9 AM

```
0 9 * * 1 /home/user/report.sh
```

Run at system reboot

```
@reboot /home/user/startup.sh
```

6. Special Symbols

Asterisk `*`

Means “every”

```
* * * * *   → every minute
```

Step `*/n`

Means “every n units”

```
*/10 * * * * → every 10 minutes
```

Range `a-b`

```
1-5 * * * * → minutes 1 to 5
```

List `a,b,c`

```
0 9 * * 1,3,5 → Mon, Wed, Fri at 9 AM
```

7. Predefined Macros

Instead of writing full expressions:

```
@yearly     → 0 0 1 1 *
@monthly    → 0 0 1 * *
@weekly     → 0 0 * * 0
@daily      → 0 0 * * *
@hourly     → 0 * * * *
```

Example:

```
@daily /home/user/daily_job.sh
```

8. Environment and Execution Context

A common mistake:

> "My script works in terminal but fails in cron."

Why?

Key differences:

1. Minimal environment

- No `.bashrc` or `.profile`
- PATH may be limited

Fix:

```
PATH=/usr/local/bin:/usr/bin:/bin
```

2. No interactive shell

- No prompts
- No user input

3. Working directory

Default is usually the user's home directory.

Fix:

```
cd /project && python train.py
```

9. Logging and Debugging

By default, cron does not show output.

Redirect output:

```
* * * * * /script.sh >> /tmp/log.txt 2>&1
```

Explanation:

- `>>` append output
- `2>&1` redirect errors to output

Example with logging

```
0 3 * * * /backup.sh >> /var/log/backup.log 2>&1
```

10. Permissions and Security

- Ensure scripts are executable:

```
chmod +x script.sh
```

- Use absolute paths:

```
/usr/bin/python3 script.py
```

- Be careful with:
  - `sudo`
  - sensitive credentials

11. System-wide Cron

There are also system-level cron configurations:

`/etc/crontab`

Format includes user:

```
* * * * * user command
```

Cron directories:

- `/etc/cron.daily`
- `/etc/cron.hourly`
- `/etc/cron.weekly`

- `/etc/cron.monthly`

Drop scripts directly into these folders.

12. Practical Use Cases

1. Machine Learning Pipeline

```
0 2 * * * python train.py
```

2. Data Collection

```
*/10 * * * * python scraper.py
```

3. System Monitoring

```
* * * * * bash check_cpu.sh
```

4. Auto-clean temporary files

```
0 0 * * * rm -rf /tmp/*
```

(Be careful with destructive commands.)

13. Common Pitfalls

1. Relative paths

Bad:

```
python script.py
```

Good:

```
/usr/bin/python3 /home/user/script.py
```

2. Missing permissions

3. Silent failures (no logs)

4. Time zone issues

Check system timezone:

```
timedatectl
```

14. Summary

- `cron` is a **time-based scheduler**
- `crontab` defines jobs using a **5-field time format**
- Always:
  - use absolute paths
  - handle logging
  - consider environment differences

Summary: These exercises build upon each other, moving from basic remote log forwarding (Ex.4) to intelligent, selective log processing with custom scripts (Ex.5), and finally to building a scheduled, automated monitoring solution based on log analysis (Ex.6).